GELIE Andréa

Battleship Game Development in Python

INTRODUCTION 4
GAME PRESENTATION 4
Game Overview : 5
Game Rules : 5
KEY FEATURES OF THE PROGRAM 5
VERSION 1 : 6
Code Architecture 6
Code Explanation 7
VERSION 2 : 7
Implemented Improvments 8
Code Explanation 8
CHALLENGES ENCOUNTERED 9
CONCLUSION 9

INTRODUCTION

This project focuses on the development of several digital versions of the classic Battleship game using Python. Battleship is a strategic game in which two players compete to sink all of the opponent's ships on a hidden grid. The main objective of this project was to apply and strengthen programming skills in Python while exploring concepts such as game logic implementation, user interaction management, error handling, and data structure manipulation. This report presents the overall game mechanics, the architecture of the program, the main technical challenges encountered during development, and the solutions implemented to improve the functionality and reliability of the game.

GAME PRESENTATION

Game Overview :

Battleship is a strategy game in which two players attempt to destroy all of the opponent's ships. Each player begins the game with a hidden 10x10 grid on which ships are placed strategically. The position of the ships remains unknown to the opposing player throughout the game

At the beginning of the match, each player places their ships either horizontally or vertically on the grid. Once all ships have been positioned, players take turns attacking the opponent's grid by entering coordinates such as "A5". After each shot, the opponent indicates whether the result is: "Miss" if no ship is present "Hit" if a ship has been damaged "Sunk" if the entire ship has been destroyed The game continues until one player successfully sinks all of the opponent's ships.

Game Rules :

- **Ship placement :** Ships must be placed on a 10x10 grid without overlapping. They can only be positioned horizontally or vertically.

- **Fleet composition :** The game includes several ships of different sizes, such as an aircraft carrier, submarine, destroyer, torpedo boat, and small boat.

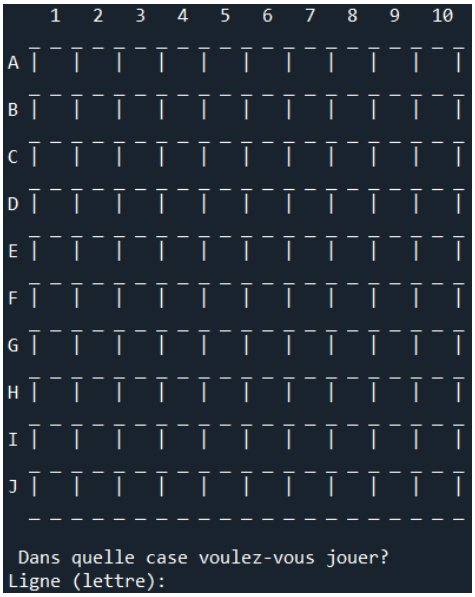
- **Turn-based gameplay :** During each turn, a player selects a coordinate to attack on the opponent's grid.

- **End of the game :** The first player to sink all enemy ships wins the game.

KEY FEATURES OF THE PROGRAM

The Battleship project includes several important functionalities:

- **User Interface :** A console-based interface allowing players to place ships and interact with the game grid.



- **Random Ship Placement:** The computer places ships randomly while ensuring that positions remain valid and do not overlap.

- **Shot Management:** The program checks whether a selected coordinate corresponds to a hit, miss, or sunk ship.

- **Error Handling:** Invalid actions, such as repeated shots or incorrect coordinates, are automatically detected and rejected.

- **Endgame Detection:** The game automatically detects when all ships have been destroyed and announces the winner.

VERSION 1 :

Code Architecture

The program is divided into several core functions:

- **PlacerBateauOrdi() :** Handles the random placement of the computer's ships while validating positions.
- **InitialiserEcran() and afficheGrilleOrdi() :** Responsible for displaying the game grids.
- **AfficheEcran() :** Updates and displays the results of each shot.
- **JouerJoueur() :**Manages player actions, evaluates shots, and updates the game state.

The program relies on NumPy arrays to represent the game grids and manage game progression efficiently

The following numerical values are assigned to the ships:

- 0 : water
- 1 : small boat
- 2 : torpedo boat
- 3 : submarine
- 4 : destroyer
- 5 : aircraft carrier

Code Explanation

The variable i represents the size of the ship currently being placed. The coordinates **a** and **b** correspond to randomly generated positions on the grid. The condition **b + i - 1 <= 9** ensures that the ship remains within the boundaries of the grid. The condition **A[a][b + k] == 0** verifies whether a grid position is empty and available for placement. If all required positions are valid, the ship is successfully placed on the grid. Otherwise, the placement process restarts until a valid position is found.

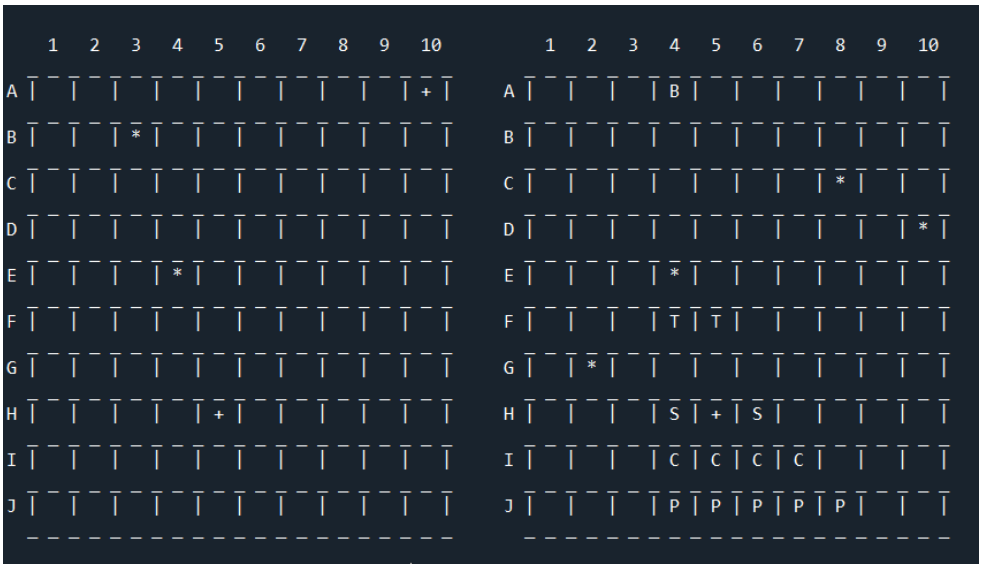
VERSION 2 :

Implemented Improvments

Several improvements were introduced in the second version of the project.

First, I implemented the function **AffricheGrilleJoueur()**, which displays the player's grid with ships represented using letters.

I also developed the function **afficheEcransDouble()**, which simultaneously displays both the player's and the computer's grids while hiding the opponent's ship positions.



Finally, the gameplay was improved by introducing alternating turns between the player and the computer. While the player selects attacks manually, the computer performs random attacks on the player's grid.

Code Explanation

In the function PlacerBateauJoueur(), I applied the same validation logic used in PlacerBateauOrdi() in order to reduce the risk of errors and maintain consistency between both versions of the game. In the function Jouer(), ships are now passed as arguments because the game manages two separate players, each with their own fleet. This approach allows more precise tracking and management of ships during gameplay.

CHALLENGES ENCOUNTERED

First challenge :

One of the main difficulties encountered during development was correctly displaying “X” symbols on positions that had previously contained “+” symbols. Initially, for debugging purposes, I displayed the computer’s ship positions directly on the grid.

However, when a ship was hit, its position was replaced with the value 0, causing the program to lose the original coordinates of the ship. To solve this issue, I stopped replacing ship positions with 0 after they had been hit, allowing the program to preserve the ship coordinates throughout the game.

Second challenge :

Another issue involved preventing players from attacking the same position multiple times. To address this problem, I created a second matrix called **JeuInitia**l, which stores the original state of the game. This allowed me to modify the active game matrix while preserving the original ship positions.

CONCLUSION

This project provided an excellent opportunity to strengthen my Python programming skills while developing a complete interactive game application. Beyond the implementation of the Battleship game itself, the project helped improve my understanding of algorithmic logic, grid manipulation, error handling, and game state management. The different improvements introduced throughout the project significantly enhanced both the reliability of the program and the overall gameplay experience.